

Exercise 07

Building a Multi-File Project

For this exercise, you get to fill in a few missing parts in a program that's made from four implementation files and three header files. Most of the code is already written for you. You get to write two of the source files and a simple Makefile to help automate an efficient build for the program.

On the course homepage, you'll find five source files, two sample input files and two expected output files for this exercise. You can download the files from the web page, or you should be able to use the following shell command to copy them to the current directory if you're working on remote.eos.ncsu.edu.

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise07/* .
```

Program Behavior

The job of this program is to read a list of names and a table of floating-point values from standard input and print a report to standard output. The input will look like the following example (input-1.txt in the starter files). It starts with two integers. The first gives a number of events. This is the number of rows in the table of values. The second integer gives the number of names, which is also the number of columns in the table. Each name can be up to 12 characters in length. There can be any number of names, and the table can have any number of rows and columns. The example below has a list of four names, followed by a table of three rows and four columns.

```
3 4
Angie Bryan Cortez Deanne
3.4 5.2 3.8 2.0
3.1 4.9 5.0 3.2
3.7 4.7 0.0 6.4
```

Each row of the table represents an event (e.g., a basketball game, a final exam or a trivia contest). The columns represent scores for each of the names at that event. In the input above, for example, Angie got a score of 3.4 on event 1 and Deanne got a score of 6.4 on event 3.

When it's complete, you should be able to run the program on the contents of the input-1.txt file and get the following output. This output shows a header at the top, and each subsequent row reports scores for one event. The report starts with an event number printed right-aligned in a 5-character field (starting from event 1). This is followed by the average score for that event, reported right aligned in 5-character field, rounded to two fractional digits. For example, 3.60 is the average score for the values on the first row of the table, 3.4, 5.2, 3.8 and 2.0. The next column contains the name with the highest score for that event in a 12-character field, then a space, then the score for that name in a 5-character field rounded to two fractional digits. The last column contains the name with the lowest score for that event in a 12-character field, then a space, then the score for that name in a 5-character field rounded to two fractional digits. For the first row of the table, 5.2 is the highest score. That's the score for Bryan. The lowest score is 2.0, for the name Deanne. If there are ties for the highest (or lowest) score, you can report any of the names tied for highest (or lowest). The sample inputs don't have any cases like this and we won't test your solution on cases where the results are ambiguous.

```
$ ./main < input-1.txt
event | avg | high | low
-----+-----+-----+-----
  1 | 3.60 | Bryan 5.20 | Deanne 2.00
  2 | 4.05 | Cortez 5.00 | Angie 3.10
  3 | 3.70 | Deanne 6.40 | Cortez 0.00
```

The figure below illustrates the formatting of the table header and the fields within the table. Columns are separated by vertical bars, with an extra space on either side of the vertical bar. The bottom line of the header is

made of dashes and plus signs.

6 dashes		7 dashes		20 dashes		19 dashes	
event		avg		high			low
1		3.60		Bryan	5.20	Deanne	2.00
2		4.05		Cortez	5.00	Angie	3.10
3		3.70		Deanne	6.40	Cortez	0.00

5 characters wide.

12 characters wide.

If you want to test your program against the expected output files, you can capture its output to a file, then use diff to make sure your output matches the corresponding expected output file exactly. This can help you make sure your table format is right. Expected output files are provided for input-1.txt and input-2.txt.

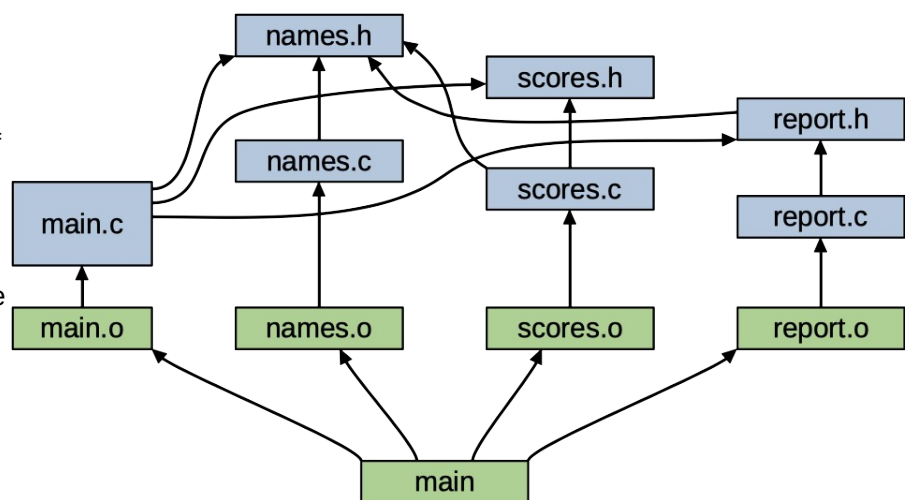
```
$ ./main < input-1.txt > output.txt
$ diff output.txt expected-1.txt
```

Program Organization

The figure below shows the organization of source files in this project. This will help you see what your Makefile needs to do to build the executable program, and what files need to be rebuilt when one of the source files is modified. Most of these source files are provided with the starter. You just have to write the **report.h** header file and the **report.c** implementation file.

The arrows at the top of the figure show which header files are included by implementation files. For example, main.c uses functions from all three of the other components, so it includes their header files.

The green boxes in the lower part of the figure show files that your makefile needs to be able to build. Each of the object files (the ones ending in .o) are built from the corresponding implementation file. The main executable is built by linking all the object files together.



Implementing Report

You'll need to create the report.h header file and the report.c implementation file. The report component just needs one function, named report(). It's the one that prints out the table with the report of average, high and low scores for each event. If you look at the main.c file, you'll see that this function is called from main() just before the program exits. Your report() function will need to take the four parameters passed to it from this call. The first two parameters tell you the size of the array of names and the size of the 2D array of scores. The next two parameters give you the names array and the scores array so your function can print out the table described above.

You'll need to define the report() function in the implementation file, and you need to give a prototype for it in the report.h header. That way, the main component will know about this function before it tries to call it. Your report() function will need to use the NAME_LIMIT constant defined in the **names.h** header. You'll need it to specify the

size of the strings in the array of names. To use this constant in your `report()` prototype, you'll need to include **names.h** from your header. The picture above shows this with an arrow from **report.h** to **names.h**.

Notice that the **names.c** and **scores.c** files include their own header file as its first line of code. It's a good practice for every component to include its own header first like this. It helps make sure that the header compiles cleanly (without depending on inclusion of other headers first), and it helps detect inconsistencies in case prototypes and external variable declarations in the header don't match the definitions in the implementation file. If you had a type mismatch between what the header declared and what the implementation file actually defined, the compiler would complain about it. You'll want to do the same with your **report.c** implementation file.

Once you have these files written, you should be able to compile your program using a command like the following. This compiles all four source files and links them together into an executable named **main**.

```
gcc -Wall -std=c99 -g main.c report.c scores.c names.c -o main
```

Build Automation

You will need to create your own Makefile to build this project from the seven source files. Your Makefile should be named "Makefile", with a capital M at the start and no filename extension at the end of the file. Some text editors like to try to put a filename extension at the end, so be sure your editor doesn't name this file something different from what we're expecting.

Your Makefile should know how to rebuild each target file independently, and it should have rules that rebuild each target whenever one of its prerequisites change. Building the project in individual components like this is a good idea. If one source file changes, we only need to rebuild the targets that depend on it; you don't have to rebuild everything.

Your Makefile will need five rules that describe targets to build and the prerequisites they depend on.

- Your first one will say that the executable named **main** depends on object files for every one of the four components. This will tell make that the executable needs to be rebuilt whenever one of the object files is changed. For this rule, you can give your own gcc command for linking the four object files into an executable, or you can use make's built-in rule for making an executable from a collection of object files.. Putting **main** as the first target in your makefile makes it the default target. If you just enter "make" with no other arguments, it will try to build this target, automatically building its prerequisite objects if needed.
- You will need four more rules in your makefile, one for each component. Each of these rules will build an object file for the component from the component's source file. The prerequisites for one of these rules will need to list the component's implementation file, along with any of the headers it includes, either directly or indirectly. Listing all these source files in the prerequisites will let make know that it needs to rebuild the object file if any of these files is changed.

The arrows in the figure above help to show the dependencies between files. For example, if **main.c** is changed, the **main.o** object file will need to be rebuilt, then the object files should be linked to recreate a new executable. If **report.h** is modified, both **main.o** and **report.o** will need to be rebuilt (since **main.c** and **report.c** both include this header) and then the executable will need to be rebuilt from all the object files. All the implementation files include **names.h**, so, if it changes, every object will need to be rebuilt and then the executable needs to be built by linking them together.

Built-in Rules

In your Makefile, you can give rules with compile instructions for the target, or, you can use make's default rules to do most the work for you. If you use the rules built into make, you'll need to set the `CFLAGS` variable to contain the options we normally use during compilation (`-Wall`, `-std=c99` and `-g`), and you'll need to redefine the `CC` variable so make uses gcc rather than cc (although these two names actually get you to the same compiler on the common platform systems). Even if you plan to use the built-in rules for compiling and linking, you still

need some dependency lines, to tell make what needs to be built and what prerequisites each target depends on.

In my solution, I used make's rules to do most of the work for me. My Makefile ended up being simple, with only 7 non-empty lines. I had five rules for building targets and two more lines to set variables used by the built-in rules.

Testing your Makefile

Once your makefile is working, you should be able to build the project like the following example. From the output of make, you can see that it's compiling each source file separately (the first four output lines), then linking them together into an executable. If you've previously built your program, make may skip some of these steps, if, for example, one of the object files is newer than any of the source files it depends on or if the executable is newer than the object files it depends on.

```
$ make
gcc -Wall -std=c99 -g -c -o main.o main.c
gcc -Wall -std=c99 -g -c -o report.o report.c
gcc -Wall -std=c99 -g -c -o scores.o scores.c
gcc -Wall -std=c99 -g -c -o names.o names.c
gcc main.o report.o scores.o names.o -o main
```

If you try to run make again right away (without changing any of your source files), you can see that it tries to be smart about what it rebuilds. If no source file has changed, it doesn't need to rebuild any of the targets:

```
$ make
make: 'main' is up to date.
```

You can check that your makefile is set up to rebuild your program selectively. Modify one of the source files (e.g., temporarily add a blank line somewhere and save) and make sure it just rebuilds the parts that need to be rebuilt. For example, if I make a change to my **main.c** file, it just has to rebuild the object file for main and then relink to update the executable.

```
$ make
gcc -Wall -std=c99 -g -c -o main.o main.c
gcc main.o report.o scores.o names.o -o main
```

Now, try modifying the **report.h** source file. Both main.c and report.c include this header, so, after it changes, make will need to these two objects, then relink to rebuild the executable.

```
$ make
gcc -Wall -std=c99 -g -c -o main.o main.c
gcc -Wall -std=c99 -g -c -o report.o report.c
gcc main.o report.o scores.o names.o -o main
```

If you modify the **names.h** header and then rebuild, the makefile will have to recompile every implementation file, then re-link.

Submitting your Work

When you're done, use the exercise_07 assignment on Moodle to submit three files, **report.c**, **report.h** and your **Makefile**. You don't need to submit the other source files. You shouldn't need to modify the others, so we can just use the versions from the starter when we're grading your solution.

Be sure not to put your files in a zip file or some other type of archive when you submit them. Just add each of the three files above to your submission. Exercise 5 describes how to do this in a little more detail.